

Final Project Report: xv6 Lottery Scheduler

Name: Michael Lartey

Course: Operating Systems

Project: xv6 Lottery Scheduler

Introduction

The purpose of this project was to modify the xv6 operating system and implement a Lottery Scheduler. Lottery scheduling is a proportional-share CPU scheduling algorithm where processes are assigned tickets. Processes with more tickets have a greater probability of being selected to run by the CPU scheduler.

Project Objectives

This project helped develop an understanding of xv6 kernel structure, CPU scheduling algorithms, system calls, and process management. The main objective was to replace the default scheduler with a lottery-based scheduler and analyze its behavior.

System Calls Implemented

Two new system calls were implemented for this project. The first system call, `settickets(int number)`, allows a process to change the number of tickets it owns. The second system call, `getpinfo(struct pstat *)`, returns information about processes such as process ID, ticket count, and how many times the process has been scheduled.

Kernel Modifications

Several kernel files were modified to support lottery scheduling. New fields named `tickets` and `ticks` were added to the process structure inside `proc.h`. The scheduler inside `proc.c` was modified to calculate the total number of tickets among runnable processes, generate a random winning ticket, and schedule the selected process. Child processes were also updated to inherit ticket values from their parent processes.

Scheduler Logic

The lottery scheduler works by first calculating the total number of tickets owned by runnable processes. A pseudo-random number generator selects a winning ticket. The process whose ticket range contains the winning ticket is scheduled to run. Each time a process is selected, its tick counter is incremented to track how many times it has executed.

Testing and Results

The scheduler was tested using three processes with ticket ratios of 30, 20, and 10. The `getpinfo()` system call was used to collect scheduling statistics. The results showed that the process with the highest ticket count received the most CPU time, while the process with the fewest tickets received the least CPU time. This confirmed that the lottery scheduler was working correctly.

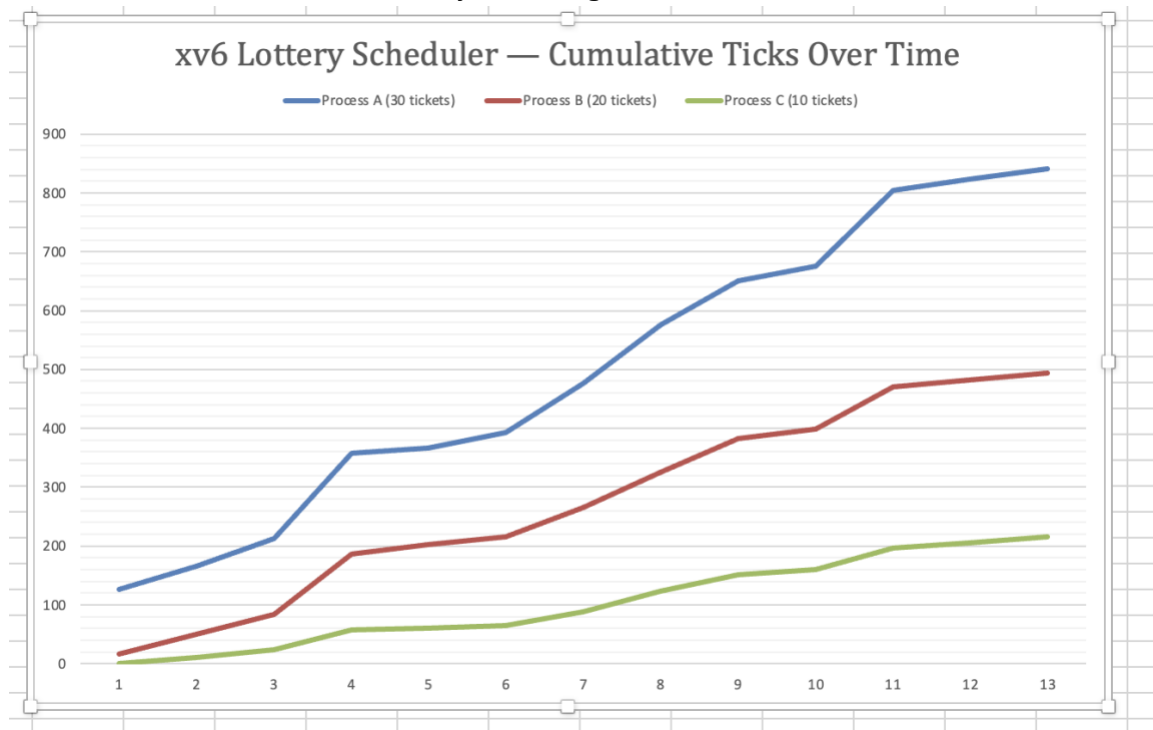
```
failed
$
Booting from Hard Disk..xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bnap8
init: starting sh
$ testlottery

=== Lottery Scheduler Results ===
Total ticks: 553
Process A (30t): 248 ticks (44%)
Process B (20t): 210 ticks (37%)
Process C (10t): 95 ticks (17%)
Expected: A-50% B-33% C-17%
=====
$ graphtest
sample,ticks_A,ticks_B,ticks_C
1,126,17,0
2,166,50,11
3,213,84,24
4,357,187,57
5,366[A02,60
6,393,215,65
7,476,266,89
8,576,325,123
9,651,383,151
10,675,399,160
11xec: fail197
exec 3,482,205
13,841,494,216
14,887,532,235
15,960,596,261
16,1003,630,281
17,1020,639,285
18,1092,681,310
19,1119,701,318
20,1133,707,320
$
```

Graph Analysis

The graph created from the collected scheduling data showed proportional CPU allocation between the three processes. The process with 30 tickets consistently accumulated more ticks over time than the processes with 20 and 10 tickets. The graph demonstrated that the

scheduler distributed CPU time fairly according to ticket allocation.



Challenges Encountered

Several challenges were encountered during development, including system call registration, kernel compilation errors, debugging scheduler logic, and adapting the implementation to xv6-riscv. These issues were resolved through debugging, restoring modified kernel files, and carefully updating the scheduler implementation.

Conclusion

This project successfully implemented a Lottery Scheduler in xv6. The scheduler distributed CPU time proportionally according to ticket allocation and demonstrated how probabilistic scheduling algorithms work inside an operating system kernel. The project provided valuable experience working with system calls, process scheduling, and low-level kernel programming.

Files Modified

- kernel/proc.h
- kernel/proc.c
- kernel/sysproc.c
- kernel/syscall.c
- kernel/syscall.h
- user/user.h
- user/usys.pl

- `kernel/pstat.h`
- `user/testsched.c`
- `Makefile`

References

1. xv6 Operating System Source Code
2. Operating Systems: Three Easy Pieces (OSTEP)
3. xv6-riscv Documentation